

Practica 05/05/2026

Luis Gálvez

Tabla de contenidos

1 Practica - 5 Variables y tipos de datos	1
1.1 Ejercicio 1	1
1.1.1 Desarrollo	2
1.2 Ejercicio 2: Trabajando con strings	3
1.2.1 Desarrollo	3
1.3 Preguntas	4
1.3.1 Desarrollo	4

1 Practica - 5 Variables y tipos de datos

1.1 Ejercicio 1

Contexto: Formas parte del equipo SIG de una corporación autónoma regional y necesitas estructurar los metadatos de una nueva estación de monitoreo ambiental antes de ingresarla a la base de datos espacial.

Instrucciones de código:

1. Define las siguientes variables respetando estrictamente el estándar `snake_case`:
 - Nombre de la estación: “Páramo de Santurbán”
 - Latitud: 7.2514
 - Longitud: -72.9069
 - Elevación en metros: 3350
 - ¿Está activa?: **Verdadero** (Usa el tipo booleano correcto según tu lenguaje).
2. Crea una estructura de colección (Lista en Python, Vector en R, o Array en Julia) que contenga la Latitud y la Longitud, en ese orden.
3. Crea un **Diccionario** (o Lista con nombres en R) llamado `metadatos_estacion` que agrupe todas las variables anteriores bajo claves descriptivas.
4. Usando la indexación adecuada (Base 0 o Base 1, según el lenguaje que elegiste), extrae específicamente la **Longitud** desde la colección de coordenadas que está dentro del diccionario y guárdala en una nueva

variable.

5. Se ha realizado una nueva medición topográfica y la elevación real es 12.5 metros más alta. Actualiza la variable de elevación utilizando el operador de incremento.
6. Imprime en consola un mensaje dinámico utilizando interpolación de texto (ej. `f-strings`, `paste()` o `$`) que diga exactamente: *“La estación [Nombre] se encuentra operativa en la longitud [Longitud] a una altura actualizada de [Elevación] msnm.”*

1.1.1 Desarrollo

```
# 1 -----
nombre_estacion = "Paramo de Santurbán"
latitud = 7.2514
longitud = -72.9069
elevacion_metros = 3350
activa = True

# 2 -----
coordenadas = [latitud, longitud]

# 3 -----
metadatos_estacion = {
    "Nombre de la estación": nombre_estacion,
    "Latitud": latitud,
    "Longitud": longitud,
    "Elevación en metros": elevacion_metros,
    "Activa": activa,
    "Lista_creada": coordenadas
}

# 4 -----
longitud_extraida = metadatos_estacion["Lista_creada"][1]

# 5 -----
elevacion_metros += 12.5

# 6 -----
print(
    f"La estación {nombre_estacion} se encuentra operativa "
    f"en la longitud {longitud} a una altura actualizada "
    f"de {elevacion_metros} msnm."
)
```

La estación Paramo de Santurbán se encuentra operativa en la longitud -72.9069 a una altura

1.2 Ejercicio 2: Trabajando con strings

Contexto: Has recibido una tabla de Excel con la toponimia de áreas protegidas de Colombia capturada manualmente por diferentes operarios. Los textos están sucios y, si intentas hacer un cruce espacial (*Spatial Join*) con la capa oficial del IGAC, el software arrojará errores de topología tabular.

Instrucciones de código:

1. Inicializa una variable con el siguiente texto crudo y problemático:
`registro_crudo = " sAnTuaRio de fAuna Y flora iGuaQue "`
2. **Utiliza el método o función nativa de tu lenguaje** (vista en la sección anterior) para eliminar los espacios en blanco accidentales al inicio y al final del texto.
3. **Usa el comando correspondiente para convertir** el texto limpio a formato de Título (*Title Case*), estandarizando así las mayúsculas y minúsculas.
4. **Emplea la herramienta de reemplazo** de texto para cambiar la letra " Y " por " y " (en minúscula), de modo que el conector gramatical sea correcto.
5. Utiliza caracteres de escape (`\n` y `\t`) para imprimir un pequeño reporte estructurado que muestre el antes y el después, similar a esto:

Reporte de Limpieza Toponímica:

```
Registro Original: " sAnTuaRio de fAuna Y flora iGuaQue "
```

```
Registro Limpio: "Santuario De Fauna y Flora Iguaque"
```

Estado: Listo para cruce espacial.

1.2.1 Desarrollo

```
# 1 -----
registro_crudo = " sAnTuaRio de fAuna Y flora iGuaQue "
```

```
# 2 -----
# Eliminar espacios al inicio y al final
registro_limpio = registro_crudo.strip()
```

```
# 3 -----
# Convertir a formato Título (Title Case)
registro_limpio = registro_limpio.title()
```

```
# 4 -----
# Reemplazar " Y " por " y "
registro_limpio = registro_limpio.replace(" Y ", " y ")
```

```
# 5 -----
# Imprimir reporte estructurado
```

```
print(
    "Reporte de Limpieza Toponímica:\n"
    f"\tRegistro Original: \"{registro_crudo}\"\\n"
    f"\tRegistro Limpio: \"{registro_limpio}\"\\n"
    "\tEstado: Listo para cruce espacial."
)
```

```
Reporte de Limpieza Toponímica:
  Registro Original: "    sAnTuaRio de fAuna Y flora iGuaQue    "
  Registro Limpio: "Santuario De Fauna y Flora Iguaque"
  Estado: Listo para cruce espacial.
```

1.3 Preguntas

1. **Sobre el Ejercicio 1:** ¿Qué índice numérico utilizaste para extraer la longitud de la colección y por qué ese número específicamente? ¿Qué error arrojaría el lenguaje si usas el índice del lenguaje opuesto (ej. usar 0 en R o 1 en Python buscando el primer elemento)?
2. **Sobre el Ejercicio 2:** En términos de geoprocesamiento de bases de datos relacionales, ¿por qué es un paso crítico y obligatorio aplicar métodos de limpieza como `.strip()` o `trimws()` antes de realizar uniones de tablas (*Joins*) basadas en nombres de municipios o regiones?
3. **Pregunta General:** Explica brevemente la diferencia fundamental entre el uso de triples comillas `"""` en Python versus su uso en Julia.

1.3.1 Desarrollo

1. La longitud se extrajo dentro de una lista contenida como valor en un diccionario, estaba relacionado a la clave "Lista_creada". La lista se creó ingresando primero la latitud que la longitud, por lo que los índices fueron respectivamente 0, 1 para cada una, debido a que se utilizó código python. De haberse usado código R, los índices serían 1, 2 respectivamente, por lo que no se encontraría el elemento 0 si se quisiese llamar la latitud usando el mismo índice.
2. Para limpiar los datos y hacerlos compatibles, de manera que las cadenas de texto realicen la unión de manera correcta solo si la evaluación de caracteres uno a uno arroja coincidencia.
3. En python permite crear cadenas de texto (o strings) multilinea, mientras que en julia funciona de forma similar, salvo que permite documentar funciones mediante docstrings